Interpreting Internal Representations of Transformer Models

Daniel[1, 2] Kiarash[1] Maria[3] Vincent[1, 2]

[1]Department of Computer Science
[2]Department of Mathematics
[3]Department of Mechanical and Industrial Engineering
University of Toronto

May 2025

UNIVERSITY OF TORONTO

Table of Contents

UNIVERSITY OF TORONTO

Deep Learning

Since 2012, deep learning architectures have achieved notable achievements in the following fields:

1. Computer vision: object detection, image classification, segmentation
2. Natural language processing: large language models (ChatGPT, Claude, DeepSeek, etc.), machine translation, coding, math
3. Reinforcement learning: robotics, self-driving cars

and many other applied fields, such as health, engineering, etc.

Introduction

1. Despite these successes, little is known about *how* deep learning models actually process and represent input data
2. We will examine how input data changes as it flows through a model, focusing on the setting of natural language processing
3. Better interpreting how these models work would potentially help us develop safer, more efficient models, and may yield insights into other fields (as many advancements in deep learning already have) such as linguistics, cognitive science, and philosophy.

UNIVERSITY OF TORONTO

Prerequisite Vocabulary: Transformers for Text Processing

- **Tokens**: basic unit of text processed by transformers. Can be words, subwords, or characters.
- **Embeddings**: map tokens to dense vectors in a high-dimensional space. These vectors capture semantic relationships: similar tokens (e.g., "king" and "queen") have similar vectors.
  - Embeddings turn discrete tokens into continuous vectors, allowing mathematical operations (e.g., $\text{king} - \text{man} + \text{woman} = \text{queen}$).
- **Positional encoding**: encoding the position of each token in a sentence with a sinusoidal function. Added to the embeddings. Allows the model to learn relative positions of tokens.

UNIVERSITY OF TORONTO

Attention is All You Need

- "Attention is All You Need" [Vaswani et al., 2023] introduced the Transformer architecture.
- Attention is a mathematical function defined as: $\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$, where
  - Q: Queries matrix. Identifies what a token seeks.
  - K: Keys matrix. Identifies what a token offers.
  - V: Values matrix. Carries the actual information to aggregate based on attention weights.
  - d_k: Dimension of keys (scaling prevents gradient saturation).
- Q, K, V: learned projections.
- Each query vector q_i computes cosine similarity with all keys k_j, creating attention weights that linearly combine value vectors v_j.

UNIVERSITY OF TORONTO

Transformer Architecture

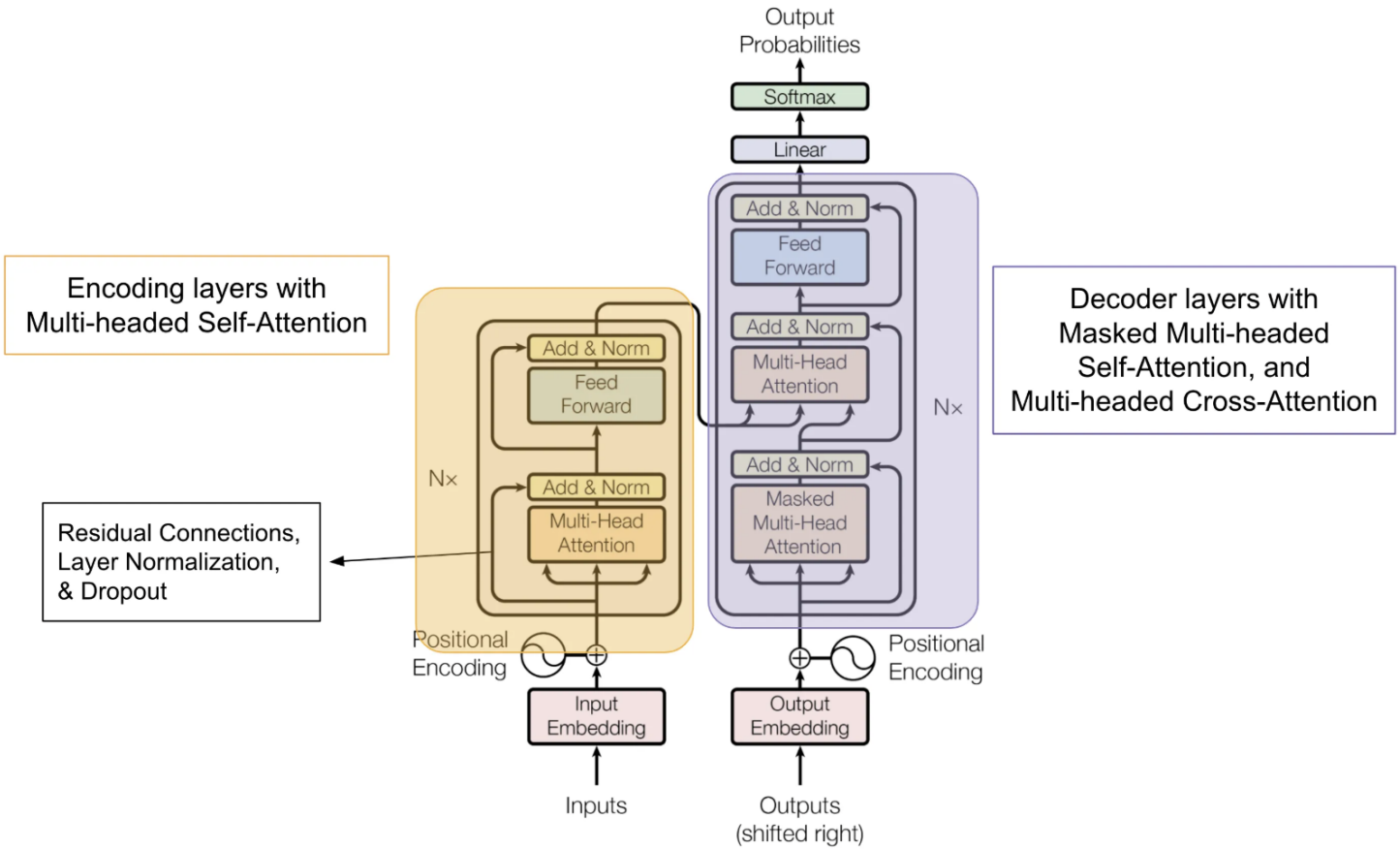

Figure: Transformer architecture.

Attention is All You Need

- Instead of one attention function, transformers use $h = 8$ parallel heads:

$$\begin{aligned}\text{MultiHead}(Q, K, V) &= \text{MultiHead}(XW_i^Q, XW_i^K, XW_i^V) \\ &= \text{Concat}(\text{head}_1, \ldots, \text{head}_k)W^0\end{aligned}$$

- Each head applies attention to linearly projected versions of Q, K, V: $\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$, where W_i^Q, W_i^K, W_i^V are learnable projection matrices.
- Each head learns a unique 'lens' to focus on different relationships (e.g., syntax vs. semantics).

UNIVERSITY OF TORONTO

Our project: Interpretability

Mechanistic interpretability: understanding the internal workings of AI models by identifiying the specific mechanisms driving their behaviour.
Why study internal model representations?

- Trust and safety: Transformers are "black boxes". Learning how knowledge is represented in them helps build trust.
- Model debugging: Understanding attention patterns reveals how models prioritize features.
- Regulation compliance: Laws (like the EU AI Act) often require explanations for automated decisions. Interpretable transformers enable compliance in sectors like healthcare or finance.

UNIVERSITY OF TORONTO

Sparse Autoencoders: Intuitive Introduction

- Transformers are powerful but as previously mentioned, they act like "black boxes" so we need to peek inside to understand them.
- Sparse Autoencoders (SAEs) reveal key features the model uses by focusing on sparse and meaningful patterns in its activations.
- Imagine a night before your exam you're trying to memorize the whole textbook. You might highlight key sentences and ignore the rest. This is similar to how SAEs work (please don't be like SAEs in real life, the question you think can't possibly appear on your exam *may* still appear). I tend to be more like SAEs :)

UNIVERSITY OF TORONTO

Mathematical Formulation of SAEs

- An SAE has two parts: an encoder and a decoder.
- Encoder: Maps input $\mathbf{x} \in \mathbb{R}^n$ to a latent space $\mathbf{z} \in \mathbb{R}^m$ (where $m \gg n$).
- Decoder: Reconstructs $\hat{\mathbf{x}} \in \mathbb{R}^n$ from $\mathbf{z}$.
- Loss function:

$$L = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 + \lambda \|\mathbf{z}\|_1$$

- First term: Ensures $\hat{\mathbf{x}}$ matches $\mathbf{x}$ (reconstruction).
- Second term: Enforces sparsity in $\mathbf{z}$ using the L1 norm, controlled by λ.
- **Intuition: The SAE learns to represent the input with a few active features (sparse representation) while still being able to reconstruct the input.**

UNIVERSITY OF TORONTO

Metrics for Analyzing SAEs

- **Reconstruction Error**: How well does the SAE rebuild the input? Measured with mean squared error (MSE). This is the first term in the loss function.
- **Sparsity**: How few features are active? Often tracked with the L1 norm or count of non-zero elements. This is the second term in the loss function.
- **Monosemanticity**: Do features represent single concepts? Assessed via correlation or human evaluation. This is a new metric introduced in the paper.
- **Interpretability**: How well can we understand the features? This is the main focus of the paper.
- The paper adds five new metrics for auto-interpretation (will be specified with paper details later).

UNIVERSITY OF TORONTO

Examples of Auto-Interpreted Latents

- In language models, SAEs might find features like “dog,” “sarcasm,” or “positive sentiment.”
- In vision models, features could be “red circles” or “edges.”
- Paper: “Rethinking Evaluation of Sparse Autoencoders” [Source]

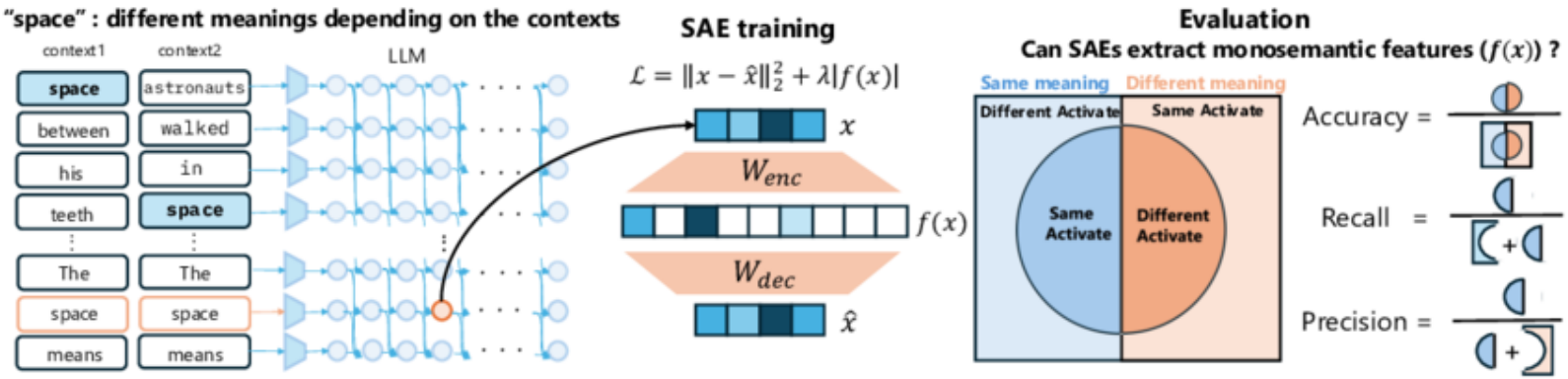

Figure: Example of SAE-identified features in an auto-interpretability pipeline.

Automatically Interpreting Millions of Features

- Paper Title: "Automatically Interpreting Millions of Features in Large Language Models." [Paulo et al., 2024]
- Tackles interpreting SAEs at scale with:
  - An automated pipeline to label SAE latents.
  - Five new scoring metrics to judge interpretation quality.
  - Open-source code and 1M+ feature interpretations.
- Figure for the pipeline/results:

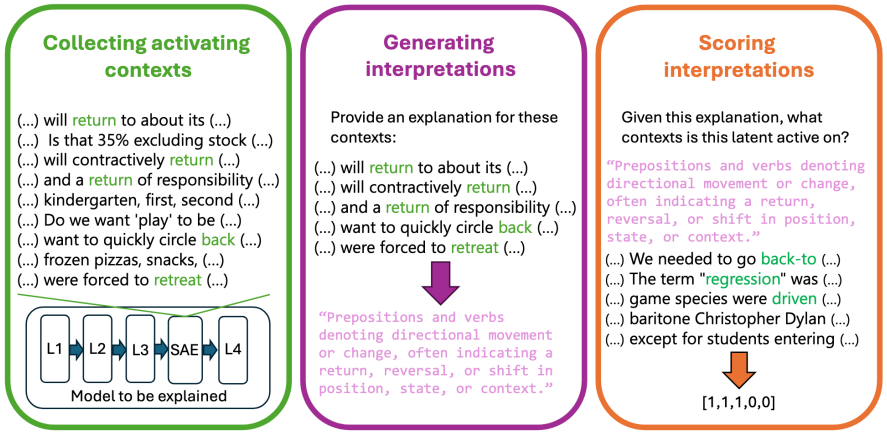

Figure: Automated interpretation pipeline. [Source]

UNIVERSITY OF TORONTO

SAEs Can Interpret Randomly Initialized Transformers

A potential counter-example for the effectiveness of SAEs:

- Paper Title: "Sparse Autoencoders Can Interpret Randomly Initialized Transformers" [Heap et al., 2025]
- SAEs should perform differently between a trained vs. randomly initalized transformer.
- Turns out that SAEs dont always do this, with similar auto-interprebility scores across randomized/ unmodified transformers
- Does this mean SAEs don't learn meaningful latent features?

UNIVERSITY OF TORONTO

SAEs Can Interpret Randomly Initialized Transformers

Transformer Model Variants Tested:

- Unmodified Trained transformer
- Randomized Transformer (incl. embeddings)
- Randomized Transformer (excl. embeddings)
- Step-0
- Gaussian Control

Setup:

- Top-k SAEs trained on 100 Million tokens from RedPajama dataset on Pythia models (70M - 7B)
- Expansion Factor of 64
- $k = 32$

UNIVERSITY OF TORONTO

SAEs Can Interpret Randomly Initialized Transformers

Evaluation:

- Fuzzing Scoring:

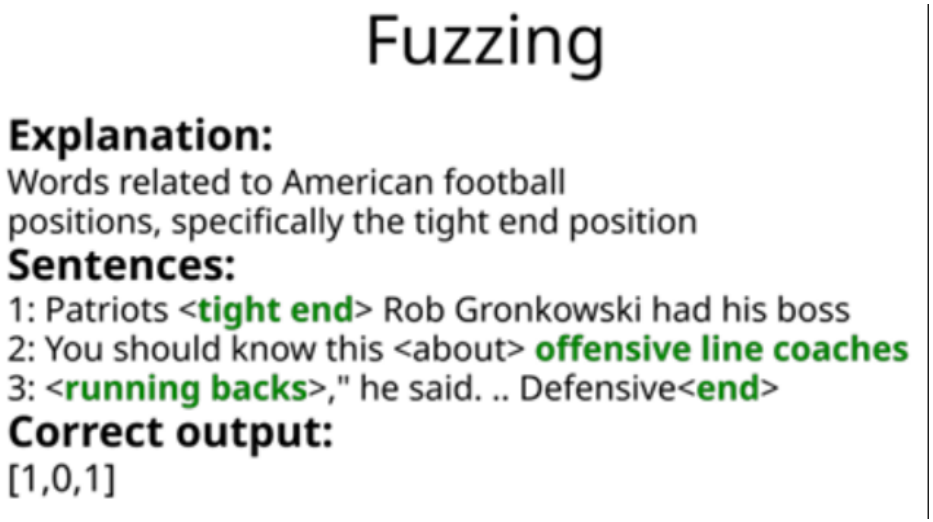

Figure: Demonstration of Fuzzing scoring.

- Randomly sampled 100 features per SAE for evalaute

UNIVERSITY OF TORONTO

SAEs Can Interpret Randomly Initialized Transformers

- Results:

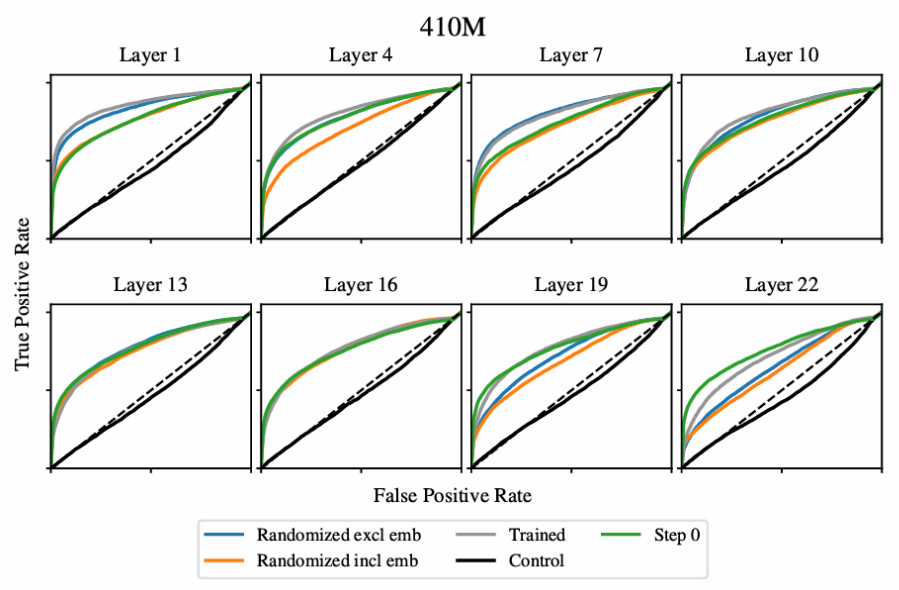

Figure: ROC curves for Pythia-410M [Source]

- Fuzzing metric fails to distinguish between trained and randomized models

SAEs Can Interpret Randomly Initialized Transformers

- Does this mean that SAEs fail at learning meaningful latent features?
- Fuzzing: measures how well learned latents work as a binary classifier
- What exactly are SAEs learning qualitatively?
- Consider the abstractness of learned features measured by entropy of observed distribution of latent activations over tokens

SAEs Can Interpret Randomly Initialized Transformers

Randomly intialized transformers preserve superposition?

- Trained model: Added layer of "semantic understanding"
- Randomized model: Preserves structure and superposition of input embeddings
- Superposition: Layer can represent more features than it has neurons - so each neuron is responsible for representing some component of some feature
- One can think of SAEs disentangling superposition!

UNIVERSITY OF TORONTO

SAEs Can Interpret Randomly Initialized Transformers

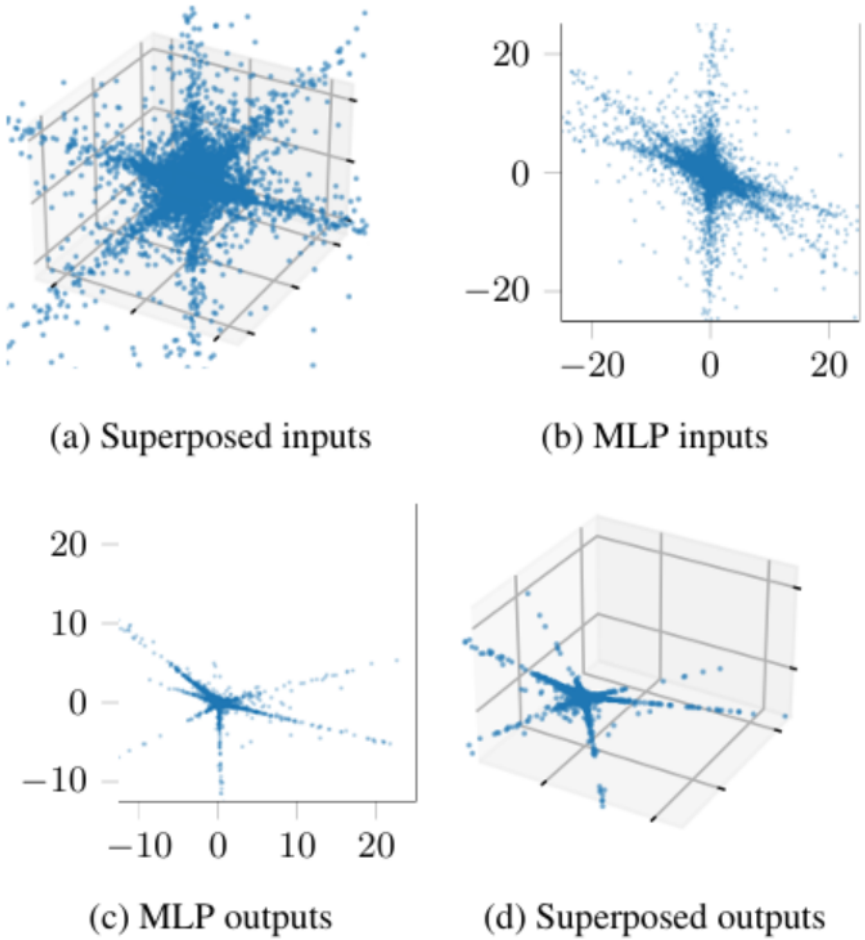

Figure: Demonstration of randomly intialized 2-layer NN preserving superposition of toy dataset generated from sampling Lomax distribution. [Source]

UNIVERSITY OF TORONTO

SAEs Can Interpret Randomly Initialized Transformers

Do randomly initialized NNs preserve or amplify superposition?

- Generate ground-truth features from hypersphere
- Then, generate datapoints by linearly combining a small number of these ground-truth features
- A synthetic dataset with known superposition properties is produced
- Feed through 2-layer NN
- Train SAEs with L1 sparsity penalty on both inputs and outputs

UNIVERSITY OF TORONTO

SAEs Can Interpret Randomly Initialized Transformers

SAE trained on inputs:

- SAEs were able to recover the ground truth features used to generate the synthetic data
- This was measured by the mean max cosine similarity between learned features and ground-truth features

SAE trained on outputs:

- Ability of SAEs to achieve low reconstruction error while maintaining high sparsity as a proxy for degree to which outputs exhibits superposition
- SAEs trained on output from superposed inputs performed better than Gaussian inputs
- Interestingly, sparsity of outputs of NN itself was consistent across superposed inputs and Gaussian control

UNIVERSITY OF TORONTO

SAEs Can Interpret Randomly Initialized Transformers

Conclusion:

- SAEs can be used on randomized models with no loss to performance according to certain metrics
- Good auto-interprebility scores does not necessarily mean that SAEs capture the underlying computational structure of the model
- Current metrics for measuring the effectiveness of SAEs fail to capture "abstractness" of SAE latents

UNIVERSITY OF TORONTO

Unanswered Questions

- Results demonstrate similar performance between the variants, but still some important differences.
  - SAE weights are different between the variants (sp. randomized vs. trained weights).
  - Greater abstraction (as measured by entropy) in the trained weights.
  - Feature explanations differ between the variants.

UNIVERSITY OF TORONTO

Our Idea: Training as we train

- **Goal**: Current work looked at the start and end of training, we examine the stages in between.
- **Hypotheses**:
  - The same metrics that were unable to distinguish between artifacts and real features, should display similar behavior over training checkpoints.
  - Superposition of input data is preserved by weight checkpoints.
  - Greater abstraction should be observed over training.

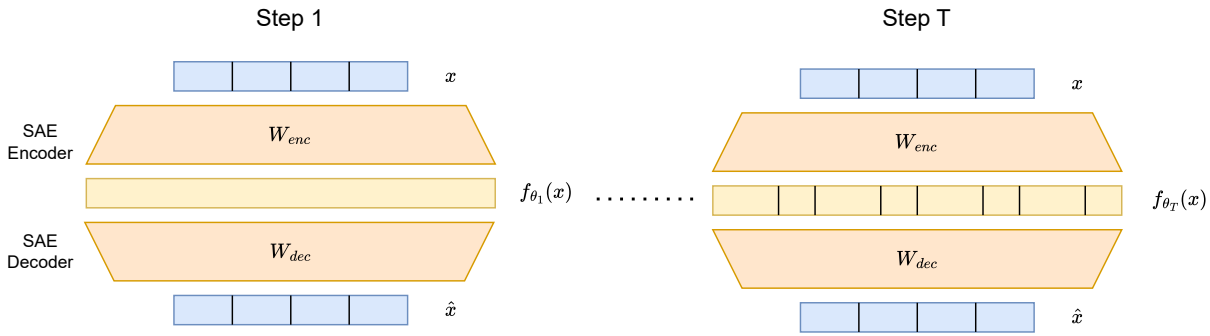

Figure: Training as we train. Our implementation is based on the *sparsify* package by EleutherAI. [Source]

Dynamical Embeddings (ongoing work)

- Currently, we require training SAEs on all layers at each checkpoint.
- Idea is to use gradient updates on model weights to evolve the activation embeddings.
- Formally, if $A(t) \in \mathbb{R}^{B \times D}$ is a layer activation batch at training iteration t; let $Z(t) = \text{Embed}(A(t)) \in \mathbb{R}^{B \times d}$ be the embedding of the activations ($d \ll D$). We want an expression

$$Z(t + \Delta t) = \text{Embed}(A(t + \Delta t)) = \text{Embed}\left(A(t) - \frac{\partial \mathcal{L}}{\partial A(t)} \Delta t\right)$$

- This can be done if we use principal component analysis (PCA) to find the embedding of the activations.

UNIVERSITY OF TORONTO

Conclusion Summary

- 5 New Scoring Metrics:
  - Detection, Fuzzing, Surprisal, Embedding, Intervention
- Important Results:
  - Fuzzing: 0.69 correlation with human eval
  - Intervention: -0.37 correlation with context scores
  - Layer 32: 40% higher scores than Layer 1
- Cost Analysis:
  - Embedding: \$50/100k latents
  - Simulation: 30x more expensive

Conclusion Takeaways

- Automated interpretations help and enable SAE debugging!
- Output features need causal metrics

UNIVERSITY OF TORONTO

SAEs can Interpret Randomly Initialized Transformers

From: [Heap et al., 2025]

Unanswered Questions

- Results demonstrate similar performance between the variants, but still some important differences.
  - SAE weights are different between the variants (sp. randomized vs. trained weights).
  - Greater abstraction (as measured by entropy) in the trained weights.
  - Feature explanations differ between the variants.

UNIVERSITY OF TORONTO

Our Idea: Training as we train

- **Goal**: Current work looked at the start and end of training, we examine the stages in between.
- **Hypotheses**:
  - The same metrics that were unable to distinguish between artifacts and real features, should display similar behavior over training checkpoints.
  - Superposition of input data is preserved by weight checkpoints.
  - Greater abstraction should be observed over training.

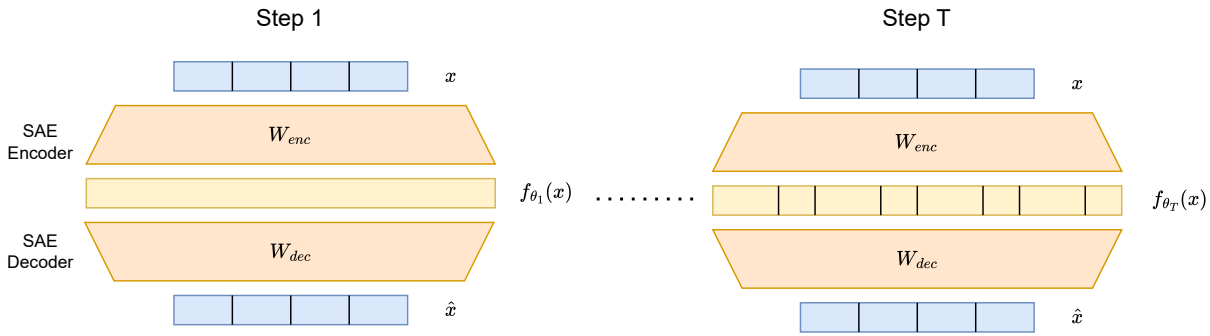

Figure: Training as we train. Our implementation is based on the *sparsify* package by EleutherAI. [Source]

Dynamical Embeddings (ongoing work)

- Currently, we require training SAEs on all layers at each checkpoint.
- Idea is to use gradient updates on model weights to evolve the activation embeddings.
- Formally, if $A(t) \in \mathbb{R}^{B \times D}$ is a layer activation batch at training iteration t; let $Z(t) = \mathrm{Embed}(A(t)) \in \mathbb{R}^{B \times d}$ be the embedding of the activations ($d \ll D$). We want an expression

$$Z(t + \Delta t) = \mathrm{Embed}(A(t + \Delta t)) = \mathrm{Embed}\left(A(t) - \frac{\partial \mathcal{L}}{\partial A(t)} \Delta t\right)$$

- This can be done if we use principal component analysis (PCA) to compute activation embeddings
[Koch and Lubich, 2007, Schotthöfer et al., 2022].

UNIVERSITY OF TORONTO

References

Heap, T., Lawson, T., Farnik, L., and Aitchison, L. (2025).
Sparse autoencoders can interpret randomly initialized transformers.

Koch, O. and Lubich, C. (2007).
Dynamical low-rank approximation.
SIAM Journal on Matrix Analysis and Applications, 29(2):434–454.

Paulo, G., Mallen, A., Juang, C., and Belrose, N. (2024).
Automatically interpreting millions of features in large language models.

Schotthöfer, S., Zangrando, E., Kusch, J., Ceruti, G., and Tudisco, F. (2022).
Low-rank lottery tickets: finding efficient low-rank neural networks via matrix differential equations.
In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A., editors, *Advances in Neural Information Processing Systems*, volume 35, pages 20051–20063. Curran Associates, Inc.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2023).
Attention is all you need.

UNIVERSITY OF TORONTO